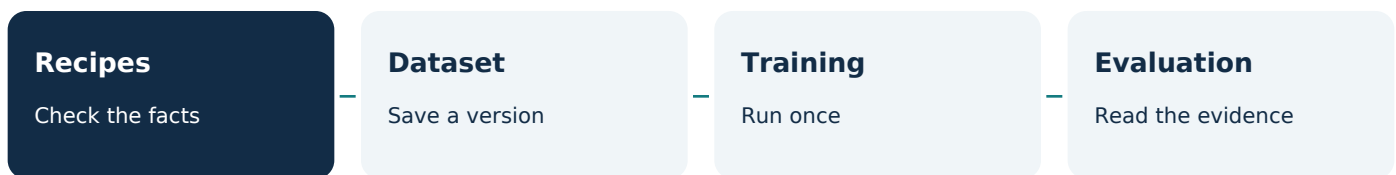


RecipeTriage AI

User guide

A step-by-step guide to using your workspace.



Edition: 12 September 2026. Based on the local source in this repository. This guide describes the current Streamlit interface; React deployment commands are identified separately. Button names follow `streamlit_ui/app.py`.

RecipeTriage organizes recipes with one or more practical labels and provides a learning workspace for building, testing and comparing small language models. A model prediction is a proposal. Human review, saved data, training results and production promotion are separate records.

Your first complete workflow: open the app, inspect a seed recipe, save a dataset version, run one baseline, train one local adapter, and compare its benchmark. Use reviewed synthetic data or DPO only after supplying the required human decisions.

This guide does not certify production readiness or claim that training improved a model. Use the actual saved evidence and the configured quality gates. Job status and provider availability change; read them in the app rather than treating a screenshot or an earlier result as current status.

Start here: sections 01-05 take you from opening the app to a saved dataset. Use section 10 for Alignment Lab and section 13 when a request fails.

Companion Markdown lessons are in the project root. The supplied `sample-recipes.json` contains the original seven unreviewed starter examples.

Find your next step

01. Open the right application	3
02. Workspace map	4
03. Recipes and the label policy	5
04. Understand one dataset record	6
05. Data Lab: draft to saved version	7
06. Dataset quality and synthetic review	8
07. Baseline and benchmark results	9
08. Train once and follow the run	10
09. Training settings and comparisons	11
10. Alignment Lab, step by step	12
11. Failure analysis and the Playground	13
12. Choose and promote a model	14
13. Troubleshooting the messages you have seen	15
14. Your next steps and quick reference	16

Click an entry to jump to that section. Page tabs and button names match the Streamlit UI inspected for this edition.

01. Open the right application

Your local project folder is `/Users/smohandoss/Documents/GitHub/recipe-triage`. Start Docker Desktop before using Docker commands on your Mac.

Existing Streamlit server setup: run these commands from the project folder. They use the private configuration you already created.

```
cd /Users/smohandoss/Documents/GitHub/recipe-triage
docker compose --env-file .server/compose.env -f compose.server.yml ps
```

If the stack is stopped, start it:

```
docker compose --env-file .server/compose.env -f compose.server.yml up -d --wait --wait-timeout 240
```

Open **<https://localhost:8543/>** for the local-check setup. Select **Check backend connection**. A successful connection checks the API and database; it does not prove that a model has loaded or that Fireworks is accessible. Do not rebuild or recreate the backend during an active job.

Deployment	Browser address when running	Configuration
Streamlit single-server local check	https://localhost:8543/	<code>.server/*.env</code> , <code>compose.server.yml</code>
React development Docker stack	http://localhost:8080/	<code>.env</code> , <code>docker-compose.yml</code>
API in the React stack	http://localhost:8000/docs	Backend port published by that stack
Standalone Streamlit preview	http://localhost:8501/	Streamlit secrets and a separately running API
Vite development server	Usually http://localhost:5173/	Host Node process and Vite proxy

These are distinct deployments. The server stack keeps FastAPI and PostgreSQL private; it does not publish port 8000 or 5432. Its database volume is also separate from the React stack's volume.

First installation only: `python3 infra/server/setup.py --local-check` creates the private server configuration. Do not run it to replace an existing `.server` directory. If old credentials are lost, restore them. For public hosting, follow `SINGLE_SERVER.md` with a real domain. Localhost uses Caddy's local certificate authority and is not a public website.

Platform note: Docker commands work in macOS, Linux and PowerShell. On Windows, use your actual project path and replace host `python3` with `py -3.12`. Setup/backup tools require Python 3.11+; host ML development uses Python 3.12.

02. Workspace map

The app opens directly with workspace tabs and page tabs. There is no RecipeTriage sign-in page. Some settings are radio buttons or selection menus. Opening a tab does not approve data or start training; pages may read records or check local capabilities.

Workspace	Pages	What you accomplish
Recipes	Library; Add recipe	Import, normalize, save, search, triage and explicitly correct recipe labels.
Data Lab	Dataset Studio; Label Editor; JSONL Preview; Chat Template Preview; Synthetic Review	Create a reproducible dataset and review proposed examples.
Training Lab	SFT Monitor; LoRA Training; QLoRA Training; Experiments	Configure training, monitor jobs and compare parameter choices.
Alignment Lab	Preference Pairs; DPO Training; GRPO Experiment; Reviewed QLoRA	Record human preferences and run isolated adaptation experiments.
Evaluation Lab	Baseline; Failure Analysis; Shortcut Tests	Measure quality, inspect errors and test title sensitivity.
Playground	Triage; Model Comparison; Token Inspector	Try recipes, compare available models and inspect tokenization.
Deployment	Measured Candidates; Model Registry	Compare export constraints and manage gated model stages.

Actions with different meanings

Action	What it establishes
Create recipe draft / Update draft annotation	Editable content; a label-editor draft exists only in this Streamlit session.
Save recipe / Save dataset version	Persistent recipe or immutable dataset snapshot. Saving does not imply human approval.
Record my decision / Save my explicit preference	Your decision about the exact displayed record or candidate pair.
Check configuration	Preflight checks, with remaining runtime checks reported separately.
Start training / Start experiment	A new job, not a progress refresh.
Refresh now / Every 5 seconds	Read saved progress without creating another run.
Apply stage change subject to quality gates	An explicit registry change, allowed only when its conditions pass.

Use the returned version IDs and run IDs as references. A green "Request completed" message says the API call returned; expand the evidence and inspect status, issues and metrics to learn what actually happened.

03. Recipes and the label policy

In **Recipes > Add recipe**, choose text, message, URL or screenshot. Paste the content or upload a PNG/JPEG/WebP image of at most 4 MB. Select **Create recipe draft**, inspect the normalized fields, correct mistakes, and choose **Save only** or **Save and run triage** before **Save recipe**. Free-form extraction uses the configured hosted extraction path; inspect the normalized draft before saving.

In **Library**, search recipe content or filter by labels. All selected filter labels must match. Expand **Correct and verify labels**, choose the correct set, give your reviewer name and recipe-grounded explanation, then select **Save my verified correction**. Editing recipe content creates a new revision; reassess its labels. Use **Load prediction and review history** to inspect earlier evidence.

Label	Meaning in this project
weeknight-30min	Known total elapsed time is at most 30 minutes, including waiting.
weekend-project	Substantial cooking work taking more than 30 minutes. Passive waiting alone is insufficient.
needs-special-equipment	Requires equipment such as a blender, food processor, pasta machine or waffle iron. Ordinary utensils do not qualify.
meal-prep	The recipe explicitly supports preparing portions ahead.
dessert	Intended as dessert; sweetness alone is insufficient.
have-most-of-this	Explicit pantry data covers at least 80% of ingredient items. Matching uses normalized ingredient strings in v1.
unclear	Evidence is insufficient or contradictory. This label must appear alone.

Multiple supported labels are allowed. Duplicate labels are invalid. Missing time does not prevent an evidence-based equipment label, but it cannot support weeknight-30min. In the UI, time 0 means unknown and is submitted as JSON `null`.

Ravioli example: the bundled recipe has a 70-minute total, a pasta machine and food processor, and instructions for freezing portions. Its proposed labels are weekend-project, needs-special-equipment and meal-prep. Prefixing the title with "Easy" must not change those recipe facts. The seed annotation is a teaching proposal, not your recorded approval.

04. Understand one dataset record

A dataset example contains recipe input plus a proposed target answer and source information. Start with the bundled Black Bean Quesadillas example (`seed-002`). Its known total is 30 minutes and its proposed label is `weeknight-30min`. Pantry inventory is unknown. Its source is recorded, and its annotation is still unreviewed.

For a complete copy-and-paste starting dataset, open `sample-recipes.json`, copy the whole file and paste it into **Dataset Studio > Raw dataset** with input format `json`. It contains the seven existing starter examples, including their source notes. This guide adds no newly generated recipes or review claims.

The complete examples are in `ml/data/seed.json`. Use all seven starter records for your first train/validation/test split; one recipe is not enough for three nonempty independent splits.

Field	Why it exists
<code>recipe.id</code>	Stable identity used in split and lineage records.
<code>title, ingredients, instructions, equipment</code>	Evidence the classifier sees. Lists contain strings.
<code>time_minutes</code>	Known total elapsed time, or null when unknown.
<code>pantry_items</code>	Known inventory, or null when unknown. An empty list means known empty inventory.
<code>source_type, source_uri, source_notes</code>	Origin and limitations. Do not relabel generated content as personal or published.
<code>group_id</code>	Keeps known related variants together during splitting.
<code>labels, rationale</code>	Expected label set and the evidence supporting it.
<code>reviewed, reviewed_by</code>	Review status. A name must accompany a true review flag; use the review workflow to record actual decisions.

Synthetic records additionally need content-bound provenance: generation/candidate IDs, model, prompt version and hash, human review ID, approval time and approved-content hash. You should not invent these fields manually.

The prediction contract uses `labels` and `explanation`. The training formatter maps an example's `rationale` to that assistant explanation. Dataset source fields stay in lineage records rather than becoming extra fields in an inference Recipe.

05. Data Lab: draft to saved version

Follow this sequence for the starter, non-synthetic dataset:

1. Open **Data Lab > Dataset Studio**. The initial Raw dataset contains the seven seed examples. Keep input format json, split seed 42, and fractions 0.7, 0.15, 0.15 for your first exercise.
2. Choose **Validate and clean** under Action, then **Apply dataset action**. Inspect issues and duplicates. Correct missing fields, invalid labels or contradictory evidence; a completed HTTP response can still contain validation issues.
3. In **Label Editor**, select an example, edit labels/rationale, and select **Update draft annotation**. This changes only the session draft and clears its prior review claim.
4. Return to **Dataset Studio**. Validate again. Choose **Preview split and export**, then **Apply dataset action**. Inspect counts, split IDs, label distributions and warnings.
5. Choose **Save dataset version**, then **Apply dataset action**. Record the exact metadata.version beginning with v1-. This is the dataset identifier to use in training.
6. Open **JSONL Preview**. Select train.jsonl, validation.jsonl, test.jsonl or a .chat.jsonl counterpart. Use **Download selected file** as needed.
7. Open **Chat Template Preview**, select a recipe from the current draft, and select **Preview chat template**. Read the System rules, User recipe and Assistant expected answer, followed by model-specific markers.

If JSONL Preview is empty: first create a split preview, or choose a Saved dataset version and select **Load saved dataset**. This loads export files. In this UI, it does not replace the working draft used by Label Editor and Chat Template Preview. To edit a saved version's examples, select/download examples.json, paste its contents into Dataset Studio's Raw dataset, and validate before editing.

Two formats: JSON is one complete document, such as an array of examples. JSONL is one complete JSON object per line, without an enclosing array or commas between records. Chat JSONL wraps each example in messages with system/user/assistant roles. The chat preview formats data; it does not generate a new answer or train weights. Its first use can download tokenizer files.

Completion check: a version ID is visible, all three split files exist, and you can explain one example's input, target and source. A preview alone is not a saved version. Save before closing or refreshing the browser if you need to retain session edits.

06. Dataset quality and synthetic review

Training learns from the training split; validation guides checkpoint selection; the test split remains a held-out check. The separate RecipeTriage benchmark is another fixed comparison set. Do not put its answers or title variants into training.

The pipeline normalizes text, validates schemas, removes exact recipe-body duplicates and groups related examples before splitting. The split is deterministic for unchanged content, seed and settings. Rare-label allocation is approximate: seven examples cannot represent every label in every split. Semantic paraphrases can still escape exact checks; assign `group_id` to known related recipes and review overlap yourself. Conflicting labels are label noise, not useful diversity.

Synthetic Review workflow

1. Open **Data Lab > Synthetic Review**. Use **Generate or prepare review candidates** to request up to six Fireworks proposals, or **Queue original seed recipes for review** when needed. These are real generation/queue actions.
2. Select a Candidate. Read its revision, quality errors, draft, source/model/prompt evidence and review history.
3. If changes are needed, choose `edit`, enter your reviewer name, modify **Edited draft JSON**, and select **Record my decision**. The page reloads a new pending revision and refreshed checks.
4. Review that saved revision. Choose `approve` only if it is correct, or `reject` if it should not be used. Record your decision explicitly. Edits and approvals are separate submissions.
5. After reviewing the required seed records and at least one useful synthetic proposal, select **Build improved dataset from approved records**. Inspect the returned version, reviewed status and frozen holdouts.

For the current misleading-title category, the title must start with **Quick**, **Easy** or **Simple**, and the known time must exceed 30 minutes. For example, a plausible 55-minute stuffed-pepper recipe might be titled "Quick Stuffed Peppers" while retaining its actual time and method. A title fix does not prove that weekend-project or other labels are correct. Ambiguous proposals require `unclear` alone.

Automated checks also reject invalid labels, duplicates, overlap with protected data and incomplete generation. Passing checks still needs your judgment about plausibility and labels. Only explicitly approved synthetic records may enter the improved version. Generic Dataset Studio imports cannot bypass this boundary. The improved-data builder keeps original validation/test content fixed; it is not a general resplit of every approved example.

07. Baseline and benchmark results

Open **Evaluation Lab > Baseline**. Expand **Fixed benchmark and provider availability**. Choose hf-base or fireworks, keep maximum new tokens at 128, then select **Run fixed benchmark** once. Read Saved runs and wait for completion before testing the other provider.

RecipeTriage-Bench-v1 contains 10 teaching cases across normal, misleading-title, special-equipment, multi-label, ambiguous and hard categories. Its answer key is provisional. It is useful for reproducible exercises, but far too small to establish broad production quality. Keep inputs, answer key, prompt protocol and decoding settings fixed when comparing runs. The baseline protocol uses temperature 0 and disabled reasoning.

Metric	What it measures
Precision	Of predicted labels, how many were correct? $TP / (TP + FP)$
Recall	Of required labels, how many were found? $TP / (TP + FN)$
F1	Balance of precision and recall: $2PR / (P + R)$.
Macro F1	Mean F1 across labels; exposes weak performance on less frequent labels.
Micro F1	F1 after pooling label decisions; frequent labels contribute more.
Exact-label-set match	Fraction of recipes with every required label and no extra label.
JSON validity / schema validity	Parseable JSON versus the required keys, types and label rules. Neither proves correct recipe classification.
Latency	Measured response time under the recorded hardware and protocol. Compare like conditions.

Example: expected labels are {weekend-project, meal-prep} and predicted labels are {weekend-project, dessert}. $TP=1$, $FP=1$, $FN=1$, so $precision=recall=F1=0.5$. Exact set match is 0 for this recipe.

A run can finish yet contain invalid outputs. Inspect raw responses, usable/returned response counts, finish reasons and per-label scores as well as aggregate status. Keep failed provider calls as evidence. Repeatedly tuning to benchmark answers contaminates its role as a held-out comparison; choose hyperparameters using validation and reserve fresh evaluation for later confirmation.

Perplexity measures token prediction loss; ROUGE measures text overlap; GLUE, GSM8K and MMLU target other language or reasoning tasks. They do not replace label accuracy, structured-output validity and title robustness for this application. See BENCHMARK.md for the full lesson.

08. Train once and follow the run

Start with **Training Lab > SFT Monitor**. Local supervised fine-tuning here trains a LoRA adapter on a pinned base model; it does not perform full-parameter fine-tuning. The Playground instruct model and the training base model are distinct.

1. Select Training provider `local`. In **Training parameters JSON**, replace `dataset_version` with your exact saved version. The default refers to the teaching seed, not automatically your most recently saved dataset.
2. Keep method as `lora` for the first run. Choose **Check configuration** and **Submit training action**. Review issues, split counts and the teaching-draft warning if present.
3. A local preflight checks the configuration, immutable dataset and installed package metadata. It does not load weights or prove tokenizer lengths, memory availability or runtime compatibility; those checks occur during training.
4. Choose **Start training**, then **Submit training action** once. Save the returned run ID.
5. In **Saved runs**, select that run and set **Refresh saved runs** to **Every 5 seconds**, or use **Refresh now**. Individual local LoRA runs appear here too.
6. When complete, inspect loss history, the before/after benchmark and per-label changes. Download evidence. Lower training loss alone is not proof of better classification.

Phase/status	Meaning
queued / preparing	The worker validates inputs and prepares the model.
baseline	It is evaluating the unadapted base before training.
training	Initial validation or optimizer updates are running.
evaluating	The selected checkpoint is running the fixed benchmark.
completed	The pipeline finished; read the actual quality results.
failed / evaluation-blocked	Inspect the saved error and backend logs; do not assume training or evaluation succeeded.

HTTP 409 in Training or Alignment Lab: jobs share a worker. Inspect the existing run, including other labs' saved jobs, and wait before submitting another. Start training creates a new run after the worker becomes free. Refresh does not. Do not restart the backend or manually erase job records to clear a real active run.

The **Resume a saved training run** control is restricted to eligible waiting/evaluation-blocked managed Fireworks runs. It is not a general resume button for completed or interrupted local LoRA jobs.

09. Training settings and comparisons

Setting	Local default	Meaning
epochs	3	Passes through the training examples. More passes can overfit.
learning_rate	0.0002	Size of parameter updates, not inference temperature.
sequence_length	768	Total prompt, target and end-token budget. Overlength local examples are rejected.
batch_size	1	Examples per microbatch.
gradient_accumulation_steps	2	Microbatches accumulated before an optimizer update; effective batch is usually 2 on this single process.
seed	42	Reproducibility setting; also retain model revision, software, data and hardware.
lora_rank	8	Capacity of the learned low-rank matrices. Supported values: 4, 8, 16, 32.
lora_alpha	null, resolved to 2 x rank	Scaling for the LoRA update; default resolved value is 16.
lora_dropout	0.0	Dropout on the adapter input during training.
lora_target_policy	query-value	Selects inspected projection paths. Other policies include attention and attention-and-mlp.
gradient_checkpointing	true	Recomputes activations to reduce memory, usually adding work.

Local SFT learns from the assistant answer and selects a checkpoint using validation loss. Its before/after benchmark stays fixed. The saved run settings are authoritative if you changed defaults.

LoRA Training: inspect valid model targets, then choose ranks such as 4 and 16. The adapter learns a small update while base weights stay frozen. Compare trainable parameters, time, memory and benchmark scores.

QLoRA Training: compare LoRA and QLoRA with the same data and budget. The current QLoRA path uses real NF4 storage with CPU computation. Read its capability result; unsupported execution is reported rather than silently replaced.

Experiments: vary learning rate and rank while fixing the dataset, benchmark and other settings. The JSON `mlflow` option controls optional tracking export. Inspect the saved comparison, including regressions.

Fireworks SFT: select fireworks and check configuration first. You need an accessible tunable model, account and deployment shape. Hosted training can incur charges and has provider-specific limits. See SFT.md and EXPERIMENTS.md for the detailed lessons.

10. Alignment Lab, step by step

Alignment adjusts model behavior after an initial supervised model exists. Its four pages have different prerequisites; viewing a page does not fulfill them.

Preference Pairs

1. If no pairs exist, wait for the shared worker to become idle, then select **Generate candidate pairs** once. This needs the saved reference adapter and Fireworks access. Generation compares hosted and current QLoRA outputs; A/B order varies.
2. Select a recipe. Read both raw candidate responses against its time, ingredients, equipment and instructions. Judge correct labels, valid structure, appropriate uncertainty and resistance to misleading titles.
3. Choose a, b, tie or neither. Enter **Your reviewer name**, explain your choice if helpful, and select **Save my explicit preference**.
4. Use **Refresh pairs** and review another recipe. The choice is tied to the displayed content hash and is immutable once saved.

Choose Tie when there is no preference, and Neither when neither answer is acceptable. Do not invent an A/B preference to unlock training. Ties and Neither are stored, but excluded from DPO. DPO requires at least three independent eligible recipe groups with explicit A/B choices, distinct nonempty candidate outputs and complete generations. Multiple choices about the same underlying recipe do not meet that independence requirement.

DPO Training: review the configuration and select **Start DPO experiment** only after those requirements are met. The trainer increases the relative likelihood of chosen answers versus rejected answers, using a frozen reference adapter. It benchmarks the resulting adapter and measures desired behaviors. The current default is a small two-step learning experiment; evaluate its result before relying on it.

GRPO Experiment: inspect Reward components, then run only when you intend to perform this isolated experiment. Reward includes label F1, JSON/schema validity, appropriate uncertainty and correct counterfactual consistency. This is a fallible measurable proxy, not your human preference. The tiny implementation samples two completions per group; its current GRPO path uses no KL penalty. It cannot automatically replace production.

Reviewed QLoRA: first build a reviewed, frozen-holdout dataset through Synthetic Review. Select its Approved dataset version and **Retrain QLoRA and evaluate shortcut accuracy**. This compares the improved-data result with the reference on unchanged evaluation data. Saved jobs and evidence appear below the page.

If an Alignment button reports a worker-active 409, check **Training Lab > SFT Monitor > Saved runs** as well as Alignment's Saved runs. You may review existing preference pairs while a training job runs; generating new pairs or starting an alignment experiment must wait.

11. Failure analysis and the Playground

Evaluation Lab > Failure Analysis: select a saved Benchmark source and choose **Analyze failures**. Inspect missing/extra labels, invalid structure and ambiguity-related mistakes. Use **Load collection priorities for selected source** for suggestions about data to collect next. A failure category is a pattern and a hypothesis; it does not prove a model's internal cause.

Evaluation Lab > Shortcut Tests: select **Run shortcut and red-team diagnostics** when the worker is available. Counterfactual cases change only title adjectives while preserving time, ingredients, equipment and instructions. Compare prediction-flip rate, misleading-title accuracy and usable pair coverage together.

Prediction-flip rate = changed usable label sets / usable counterfactual pairs

A zero flip rate can still mean a model is consistently wrong. Missing or invalid answers can also hide behavior; coverage must be inspected. Collect recipes with reliable evidence around these failures, review their annotations, and keep benchmark examples out of the next training dataset.

Playground > Triage: select a seed, edit a recipe and choose provider hf, fireworks or production. For the first local request choose hf, temperature 0 and 128 maximum new tokens, then **Run inference**. Production requires an explicitly promoted, available registry version.

Playground > Model Comparison: select available models and **Start model comparison**. Compare saved side-by-side outputs, structured labels, errors and latency for the same recipe. A model being listed does not mean its adapter is installed or a deployment is ready; inspect Model availability.

Playground > Token Inspector: enter text, choose Plain text or Chat template, and select **Inspect tokens**. It shows token pieces, numeric token IDs and count. A token is not necessarily a word. Chat templates add role/boundary markers, so their token count differs from plain text.

The local Playground instruct model and the baseline/training base model are different versions of Qwen2.5-0.5B. Check the recorded model identity when comparing results. Chat preview uses the instruct tokenizer; local base training has its own recorded prompt format.

Inference computes an answer using existing weights. Training changes trainable weights using examples and a loss. Changing temperature or a title does not train the model. See [LEARNING.md](#), [FAILURE_ANALYSIS.md](#) and [ADVANCED_ANALYSIS.md](#).

12. Choose and promote a model

Measured Candidates shows saved size, resource, latency and benchmark measurements. Edit operational constraints and select **Compare candidates against constraints**. This evaluates suitability; it does not merge, quantize, promote or deploy a model. Historical measurements also do not prove large artifact files exist on the current host.

Merging and quantization are backend-host operations described in DEPLOYMENT.md. Ask the maintainer to prepare and benchmark an export before attempting to use it. The measured-candidate page does not perform those conversions. Compare quality and operational constraints together; never select solely by smallest size.

Model Registry stages

Stage	Meaning
candidate	A registered version with artifact and lineage evidence.
staging	A version admitted to pre-production checks under the configured gates.
production	The explicitly selected version used by registered-production serving.
archived	Retained history, including versions that may be eligible for rollback.

In **Model Registry**, inspect Evaluation quality gates and the selected model. Choose a Registry action, enter Your name and a reason, then **Apply stage change subject to quality gates**. Missing or failing evidence leaves promotion blocked. Training completion never automatically means production promotion.

Default gate in code	Threshold
Macro F1	At least 0.45
JSON validity and schema validity	Each at least 0.95
Misleading-title accuracy	At least 0.67
Usable counterfactual pair coverage	1.00
Prediction-flip rate	At most 0.00
F1 regression for each important label	At most 0.05 absolute versus the comparable reference

The gate also requires complete, unchanged benchmark/shortcut evidence with matching checkpoint identities and recomputed scores. MODEL_GATE_JSON can override defaults; the gate configuration displayed by your backend is authoritative.

Rollback procedure: select an archived model that previously served in production; confirm its immutable artifact files are available; choose `rollback`; enter actor/reason; submit. Current gates are checked and the production pointer changes atomically only on success. Check backend connection and run Playground inference with `production` afterward. If no version previously served in production, there is no eligible rollback target. See MODEL_REGISTRY.md for the full procedure.

13. Troubleshooting the messages you have seen

Message or symptom	Meaning and next step
HTTP 409: training or benchmark worker active	Another run holds the shared worker, even if you clicked in Alignment Lab. Inspect Training Lab's SFT Monitor and the relevant lab's Saved runs. Wait; use Refresh rather than Start.
HTTP 409: candidate changed	The review revision is stale. Refresh the queue, inspect the new record and make a fresh explicit decision. This is a different conflict from the worker lock.
HTTP 422: Resolve quality errors before approving	Read errors above the Synthetic Review draft. Choose Edit or Reject. Save corrections, inspect the new revision and then approve separately if appropriate.
HTTP 422: Provider preflight requires Fireworks	This came from an earlier backend. Current local preflight works without Fireworks. Confirm your local source/image version and deployment, and update the backend only when workers are idle.
Draft updated for this session	Label Editor changed a temporary draft. Go to Dataset Studio, validate, preview and save a version.
Preview a split or load a saved version first	JSONL Preview has no export snapshot. Use Dataset Studio's preview action or Load saved dataset.
Request completed; response shown below	The API returned. Expand JSON and read status/issues/error; this message alone does not mean a dataset was saved or training completed.
Fireworks HTTP 404	The selected model/deployment may be unavailable or inaccessible. Check the feature-specific model setting in the correct env file and your provider access; then recreate the idle backend.
Missing .server/compose.env	You used server Compose without its configuration. Use the correct stack; first-time setup creates it, while lost existing configuration should be restored.
Cannot COPY ml/requirements-hf.lock	Check that the full ml/ source exists and the backend build context is the repository root. An incomplete download or backend-only build context omits this file.
No module named recipetriage_ml	Install -e ./ml into the notebook/kernel's actual Python environment. Ask your project maintainer to check the notebook environment; see LEARNING.md.
Local tokenizer/model unavailable	Read backend logs. Check the pinned dependencies, model download/cache, disk and memory. A healthy database does not prove model readiness.
QLoRA unsupported	Read the real CPU NF4 capability error. Install matching dependencies/platform support; do not rename an ordinary LoRA result as QLoRA.
No usable preference data for DPO	Record genuine A/B decisions on at least three eligible independent groups. Ties, Neither, identical outputs or truncated candidates cannot satisfy this gate.

If a run appears stalled, inspect its ID, phase, updated timestamp and backend logs. Model download, CPU generation and training may be slow. Do not assume a stale timestamp alone proves the worker died, and do not erase database locks/records to suppress an error. Retain the exact error when requesting help.

14. Your next steps and quick reference

Your next action depends on what you have finished:

You have finished	Next action
Editing labels	Dataset Studio: validate, preview and save a version.
Saving a dataset	Baseline: measure the untrained model.
Running a baseline	SFT Monitor: check the exact dataset version and start one local LoRA run.
Training	Inspect the complete before/after benchmark and per-label changes.
Finding repeatable errors	Review useful new data or explicit preference pairs.
Preparing a candidate	Read its evidence and current registry gates before any promotion.

For optional lessons, start with `LEARNING.md` and `DATASET_V1.md`, then `BENCHMARK.md`, `TRAINING_FUNDAMENTALS.md`, `SFT.md` and `LORA.md`. Continue to `HUMAN_LEARNING.md` for preferences and `MODEL_REGISTRY.md` for deployment gates. Teaching notebooks are under `ml/notebooks/`.

A completed beginner exercise has evidence: a saved recipe or examined seed; a dataset version with split IDs; a complete baseline; one training run with before/after benchmark; an explanation of which metrics changed. A completed alignment exercise additionally has explicit eligible decisions and its own evaluation. A production exercise additionally needs passing gates, available artifacts, access control, backups and operating checks.

Five questions to answer in your own words

1. What changes during inference, and what changes during training?
2. Why is an "Easy" title insufficient evidence for weeknight-30min?
3. How do a session draft, saved dataset version and human approval differ?
4. Why can Alignment Lab report a worker conflict while LoRA is running elsewhere, and which button checks progress without creating a new job?
5. What evidence would justify preferring a trained model over the baseline, and why are lower loss or smaller model size insufficient by themselves?

Use `USE_CASES.md` for the UC-01 through UC-15 feature map, `REVIEW_CHECKLIST.md` for acceptance checks, and `VERIFICATION.md` for dated execution evidence and limitations. This guide is an operating and learning reference, not a substitute for current run results or your own review decisions.

Stopping safely: wait for all jobs to finish, then run this from the project folder to stop the Streamlit server stack while keeping data volumes:

```
docker compose --env-file .server/compose.env -f compose.server.yml down
```

To start it again, use the start command in section 01. Avoid `down --volumes`; it deletes persisted data. For coordinated backups, restore, server logs or the optional MLflow dashboard, follow `SINGLE_SERVER.md` and `README.md`. The MLflow dashboard, when separately configured, is not the same as RecipeTriage's Experiments page and is not included in the Streamlit server Compose stack.